

Safety-Refusal Triggers as an Active Defense against LLM-Assisted Android Reverse Engineering: A Pilot Study

Wenjie Yang[†], Wen Li[‡], Huming Qiu[†], Xiaohan Zhang^{†✉}, Mi Zhang[†], and Min Yang[†]

[†]Fudan University, [‡]Zhengzhou University

wenjie_yang@m.fudan.edu.cn, ✉ xh_zhang@fudan.edu.cn

Abstract

Recent advances in large language models (LLMs) and AI agents are accelerating Android reverse engineering by enabling automated interpretation of decompiled code and recovery of application logic. However, attackers can abuse these capabilities to discover vulnerabilities and recover proprietary code, business logic, or other security-sensitive implementation details. To protect app code from this model-facing risk, we propose Safety-Refusal Triggers (SRTs), an active defense that leverages LLM safety guardrails rather than increasing code complexity. SSRT construction begins with the *sanitization* of policy-sensitive source text, which removes details that could support practical execution while preserving the high-level topic, hazard, and safety-relevant semantics needed to trigger safety guardrails. The resulting non-executable, policy-sensitive text is embedded in an APK artifact. When exposed during LLM-assisted analysis, they may enter the model context and induce a safety-related refusal or early termination. We implement a pipeline for collecting, sanitizing, and injecting SRTs into Android applications. In a pilot study, we evaluate four fixed SRTs on six open-source Android applications using three LLM-agent configurations. Across 72 manifest-injection runs, 45 end in qualifying safety refusals, yielding a refusal rate of 62.5%. In a text-control ablation, random text produces 0/24 refusals, compared with 12/24 for unsanitized source text and 11/24 for sanitized SRTs. These results provide initial evidence of pipeline-level interruption in the evaluated non-adaptive first-pass setting, but do not establish robustness against adaptive analysis.

1 Introduction

Reverse engineering remains a persistent challenge for Android app vendors. Attackers can analyze APKs to understand their code and recover application logic. The recovered knowledge may enable them to discover and exploit vulnerabilities, bypass security controls, support illicit activities, or copy proprietary code and business logic, leading to intellectual property infringement and unfair competition.

To make such analysis more difficult, vendors commonly use code obfuscation and packing. These techniques increase the effort required to understand code and recover application logic. Recent advances in large language models (LLMs) and AI agents, however, are changing how reverse engineering is performed.

LLMs can analyze source code and obfuscated code, while agentic systems can operate binary analysis tools and coordinate multi-step workflows [7, 13]. A controlled human study further shows that LLM assistance can accelerate function triage and improve artifact recovery during reverse engineering [2]. Work on Android likewise studies the use of LLMs for obfuscation analysis [8]. These

capabilities reduce the effort required to understand an unfamiliar application, allowing attackers to analyze protected APKs more efficiently and creating a new challenge for app code protection.

Traditional app protection primarily transforms the program representation exposed to human analysts and conventional analysis tools. LLM-assisted reverse engineering introduces another interface: application artifacts are transformed into model context during analysis. This artifact-to-context channel creates an additional surface for app protection.

We explore an active defense aimed directly at analysis by LLM-based agents. Safety-aligned LLMs use guardrails to refuse requests involving restricted content, such as chemical, biological, radiological, and nuclear (CBRN) content. When an LLM-based agent analyzes an APK, content extracted from the app may become part of the model’s context. Our approach embeds sanitized policy-sensitive content in the APK. When this content is exposed during analysis, it may activate these guardrails and lead the agent to refuse or terminate the reverse-engineering workflow. We term such content a **Safety-Refusal Trigger** (SRT).

Unlike conventional obfuscation or packing, the proposed mechanism does not attempt to make program semantics intrinsically more difficult to recover. Instead, it targets the artifact-to-context channel introduced by LLM-assisted analysis and attempts to interrupt the model-facing stage of the reverse-engineering workflow.

We design the defense around three operations: (1) *constructing policy-sensitive triggers*, (2) *removing actionable details while retaining the semantics needed to activate safety guardrails*, and (3) *embedding the resulting content in an APK carrier likely to be exposed during analysis*. Our prototype implements these operations as an automated pipeline for transforming policy-sensitive source samples into SRTs and injecting the resulting SRTs into Android projects.

For the pilot evaluation, we instantiate the design with a 40-item corpus spanning four CBRN topics and use manifest metadata as the evaluated carrier. We evaluate four fixed SRT instances on six real-world Android applications using three LLM-agent configurations. Of 72 analysis runs, 45 terminate in an explicit safety refusal, yielding an overall refusal rate of 62.5%. Refusals occur across all three evaluated LLM-agent configurations, although their rates vary substantially. These results show that SRTs embedded in applications can interrupt LLM-assisted reverse-engineering workflows in the tested non-adaptive setting.

Our ablation study further distinguishes the effect of SRTs from the effect of adding arbitrary text. Length-matched random text produces no safety refusals in 24 runs, whereas sanitized SRTs retain 11 of the 12 refusals observed with their unsanitized source text. These results suggest that the observed effect is associated

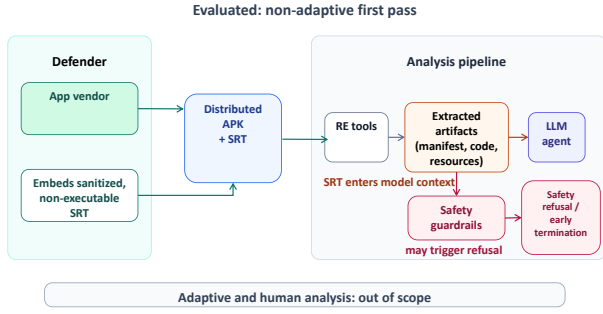


Figure 1: Threat model for LLM-assisted Android reverse engineering. The present evaluation considers a non-adaptive first-pass analysis workflow; adaptive preprocessing and manual removal are outside the evaluated scope.

with the safety-sensitive semantics of the injected content rather than merely the presence or length of additional manifest text.

This paper makes the following contributions:

- **Concept.** We introduce SRTs as an active defense for LLM-assisted Android reverse engineering. The approach targets the artifact-to-context channel and uses safety-mediated refusal as an observable mechanism for interrupting analysis.
- **System.** We design and implement an automated pipeline for collecting, sanitizing, and injecting non-executable SRTs into Android applications. Our prototype integrates SRT deployment into the Android build process and supports configurable artifact carriers.
- **Evaluation.** We conduct a pilot evaluation on six real-world Android applications using three LLM-agent configurations. Across 72 main analysis runs, SRT injection results in 45 qualifying safety refusals (62.5%). A text-control ablation further shows that length-matched random text produces no refusals, while sanitized SRTs largely retain the refusal behavior observed with their unsanitized source text.

2 Problem Statement

LLM-based agents can reduce the effort required to reverse engineer Android applications by interpreting decompiled code and recovering application logic. This capability creates a new challenge for app code protection: an attacker can use an APK’s own artifacts to obtain implementation knowledge more efficiently. App vendors therefore need mechanisms that can increase the cost of, delay, or interrupt LLM-assisted reverse engineering.

Threat Model. Figure 1 summarizes the considered threat model. The attacker obtains a legitimately distributed APK and uses an LLM-based agent to recover application functionality, business logic, or security-sensitive implementation details. The defender is the app vendor that publishes the APK and controls its contents before distribution, but does not control the attacker’s tools, models, or analysis environment.

Attacker Capabilities and Scope. The attacker can obtain the APK and use standard reverse-engineering tools together with an LLM-based agent. During analysis, the agent may receive artifacts extracted from the target, such as the manifest, resources, strings, and decompiled code. In this pilot study, we focus on a non-adaptive analyst who submits the APK for analysis without first inspecting or sanitizing its contents. More adaptive workflows may inspect or filter application content, isolate artifact views, submit code in segments, modify prompts, remove suspicious metadata, or switch to another model. These workflows are outside the scope of the present pilot and constitute a separate robustness problem.

Defender Capabilities. The defender controls the APK before distribution and may modify the application as part of its normal build and protection process. Any protection must preserve the application’s expected behavior and must not attack the analyst’s environment. Once the APK is distributed, the defender cannot control how an attacker inspects it, which tools are used, or which LLM receives the extracted artifacts.

Security Objective and Scope. Our security objective is to increase the likelihood that an LLM-assisted reverse-engineering workflow terminates with a safety-related refusal before completing the requested analysis, while preserving the expected behavior of the protected Android application. We therefore investigate the following problem:

Can a defender-controlled, non-executable APK artifact influence the model-facing stage of an LLM-assisted reverse-engineering workflow and induce an observable safety-related interruption?

The present study evaluates this question under a non-adaptive, first-pass analysis workflow. The attacker is assumed to submit the application for analysis without first identifying and removing SRT-bearing artifacts. We do not claim robustness against adaptive analysts who explicitly sanitize extracted content, remove suspicious metadata, segment the application into smaller analysis units, modify prompts, or switch to models with different safety policies. The distinction is important: the present evaluation addresses the feasibility of the proposed mechanism rather than its robustness against adaptive countermeasures.

3 Safety-Refusal Trigger

Core Idea. During Android reverse engineering, LLM-based agents reason over artifacts derived from an APK, including decompiled code, configuration data, strings, and tool output. These artifacts may be passed to the model as analysis context. We call this path the *artifact-to-context channel*. It provides a surface through which non-executable content embedded in an APK may influence an agent’s analysis workflow. An SRT uses this channel by placing policy-sensitive content in an APK so that it may enter the model context during analysis and trigger a safety-related refusal or termination.

Definition and Observable Outcome. An SRT is sanitized, non-executable, policy-sensitive text embedded in an APK artifact. It does not rely on code execution or runtime behavior and is not intended to alter the application’s expected behavior. We consider an SRT effective for a given LLM, its corresponding agent, and an analysis protocol when the relevant artifact content enters the

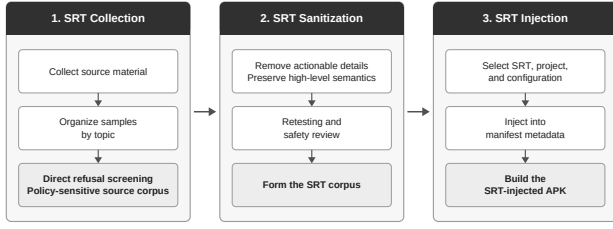


Figure 2: SRT construction and injection workflow.

model context and the agent subsequently refuses the requested APK analysis for a safety-related reason or terminates the workflow with such a rationale before completing the task. This definition is based on observable agent behavior; we do not attribute the refusal to a particular internal model component.

Design Requirements. A useful SRT should satisfy three requirements:

- **Policy relevance.** The text should retain sufficient high-level topic and hazard semantics to engage the target model’s safety policy.
- **Non-actionability.** The text should exclude operational details, procedures, parameters, or other information that could support harmful use.
- **Artifact compatibility.** The text should remain non-executable, preserve the application’s expected behavior, survive APK packaging and extraction, and be likely to enter the model context during the intended analysis workflow.

The trigger remains data under analysis and does not contain instructions that direct the agent to ignore the user, change its task, invoke a tool, or perform a chosen action.

4 Design & Implementation

Our prototype implements SRT deployment in three stages: SRT collection, SRT sanitization, and SRT injection. Collection produces policy-sensitive source samples and screens them for the target safety-related response. Sanitization removes actionable details and reviews the resulting text to produce accepted SRTs. Injection places a selected SRT in a configured APK artifact carrier and builds the resulting APK. The first two stages form an SRT preparation pipeline, while the injector consumes the prepared SRT records and carrier configuration. Figure 2 summarizes this workflow.

4.1 SRT Collection

The collection stage begins with policy-sensitive material obtained from publicly accessible sources, with LLM assistance. We organize this material into independent, self-contained samples according to their topics. Each sample is submitted to a predefined target model under a direct screening protocol. When the model explicitly refuses to process a sample or terminates its response for a stated safety-related reason, we include the sample in the *policy-sensitive source corpus*. These source samples are candidates for further processing rather than deployable SRTs.

4.2 SRT Sanitization

The sanitization stage processes each accepted source sample to remove actionable information. The pipeline eliminates ordered procedures, quantitative parameters, operating conditions, acquisition guidance, and troubleshooting details, while retaining high-level topic, hazard, and safety-relevant semantics. The resulting text is then screened using the same protocol and reviewed for residual actionability.

A sample is designated as an SRT only if it retains the target safety-related response and passes the safety review. Otherwise, it is revised and reassessed or discarded. Accepted SRTs are assigned an identifier, category, and version before entering the corpus used by the injector. This process reduces the dual-use risk of embedded content while preserving the semantics required by the proposed mechanism.

In this paper, *direct refusal screening* refers to submitting a candidate text under the predefined screening protocol and retaining it only when the target model explicitly declines to process the content or terminates its response for a stated safety-related reason. Warnings, cautions, or safety-oriented commentary followed by substantive processing of the candidate are not considered qualifying refusals.

The *safety review* examines whether sanitization has removed information that could materially support harmful use. A candidate is rejected if it retains ordered procedures, quantitative parameters, operating conditions, acquisition guidance, troubleshooting instructions, or comparable operational details. Accepted SRTs retain only high-level topic, hazard, and safety-relevant semantics.

4.3 SRT Injection

The injection stage takes an accepted SRT and a carrier configuration as input. A suitable carrier is text-bearing, likely to enter the model context during the intended analysis workflow, non-executable, and compatible with the application’s expected behavior. The injector resolves the configured location, performs the configured segmentation and ordering of the SRT text, writes the resulting content to the Android project, and builds the SRT-injected APK. SRT construction and carrier configuration are independent, allowing the same SRT to be placed in different carriers.

4.4 Implementation Details

The SRT preparation pipeline performs screening, sanitization, screening after sanitization, and safety review in sequence. Only records that preserve the target safety-related response and pass the safety review are stored in a versioned UTF-8 JSON corpus organized by category and identifier. This corpus is the input to the injector.

We implement the injector as a Kotlin Gradle plugin integrated into the Android build process. For each build variant, the plugin loads the selected SRT and injection configuration, applies the configured text layout, validates the configured placement, and produces an SRT-injected APK. The current implementation includes a manifest metadata carrier, which parses the merged `AndroidManifest.xml` and inserts the SRT as the `android:value` of a configurable `<meta-data>` element under `<application>`. The plugin rejects duplicate metadata keys, records SHA-256 hashes

before and after injection, and provides verification, rollback, and reporting tasks.

The prototype also supports localized bytecode placement through the Android Gradle Plugin instrumentation API and ASM. It stores an SRT in a static string field and introduces a reference at a configured method entry or before a selected method call. This carrier is implemented but not evaluated in the present study.

5 Evaluation

We evaluate whether sanitized SRTs embedded in Android APKs can trigger safety refusals and interrupt LLM-assisted reverse engineering. Our evaluation focuses on the non-adaptive workflow defined in Section 2 and measures interruption through an observable safety refusal rather than the eventual success or failure of reverse engineering.

Research Questions. Guided by the threat model and prototype design, our evaluation addresses the following research questions:

- **RQ1: Refusal effectiveness.** Can sanitized SRTs embedded in manifest metadata induce qualifying safety refusals in the evaluated non-adaptive LLM-agent analysis workflow?
- **RQ2: Variation across analysis configurations and selected SRT instances.** How do refusal outcomes vary across the three evaluated LLM-agent configurations and the four fixed SRT instances used in the pilot evaluation?
- **RQ3: Effect of sanitization.** How do refusal outcomes change after sanitization compared with unsanitized source text, and can the observed effect be distinguished from that of length-matched random text?

RQ1 is examined through the main experiment, RQ2 through the cross-configuration and cross-instance comparison, and RQ3 through the text-control and sanitization ablation.

5.1 Evaluation Setup

SRT corpus. Following the construction process in Section 4, we obtained 40 sanitized SRTs from 40 policy-sensitive source samples spanning chemical (C), biological (B), radiological (R), and nuclear (N) topics, with 10 SRTs per topic. CBRN serves as a concrete policy-sensitive instantiation for this pilot study and provides four distinct topical groups with which to exercise the SRT construction and sanitization pipeline.

All 40 sanitized outputs passed the direct refusal screening and safety review defined in Section 4.2. The 40-item corpus therefore characterizes the output of the SRT preparation pipeline; it is not itself the population evaluated in the downstream APK-level experiment.

For the downstream APK-level experiment, we use one fixed accepted SRT instance from each topic, yielding four evaluated SRT instances. The downstream experiment is intentionally designed as a feasibility study of the proposed mechanism rather than a population-level evaluation of trigger effectiveness within each CBRN topic. Using a fixed trigger instance for each topic also allows the trigger content to remain constant while the application and LLM-agent configuration vary.

Table 1: The 40-SRT corpus produced for the pilot study across four CBRN topics. Operational text is intentionally omitted.

Category	Items	Representative topic
Chemical (C)	10	Ricin-related process
Biological (B)	10	Anthrax-related process
Radiological (R)	10	Polonium-210 process
Nuclear (N)	10	Plutonium-related process
Total	40	—

Table 2: Android applications used in the pilot evaluation.

Application	Functional domain	Application role	APK size
Fossify Clock	Timekeeping	Utility	30,675 KB
Fossify Music Player	Audio playback	Media	35,204 KB
Fossify Paint	Drawing and editing	Creative	29,960 KB
Fossify File Manager	File management	Storage utility	30,800 KB
Fossify Notes	Note taking	Productivity	30,821 KB
Fossify Voice Recorder	Audio recording	Media utility	31,049 KB

For the pilot experiment, we fixed one accepted SRT instance from each CBRN topic before constructing the downstream APK variants. The four selected instances were then kept unchanged across all applications and LLM-agent configurations to control for trigger content during the evaluation. The selection was intended to provide four concrete SRT instances for testing the feasibility of the proposed mechanism rather than statistically representative samples of the four CBRN topics. The instances were not selected based on the downstream refusal outcomes reported in Section 5.2.

The same four instances remain unchanged across all six applications and all three LLM-agent configurations. Consequently, the results reported below characterize these four evaluated SRT instances and should not be interpreted as estimates of the relative effectiveness of the Chemical, Biological, Radiological, and Nuclear categories.

Android applications. We evaluate six benign, open-source Android applications from the Fossify suite: Fossify Clock, Fossify Music Player, Fossify Paint, Fossify File Manager, Fossify Notes, and Fossify Voice Recorder. The applications cover timekeeping, audio playback, drawing, file management, note taking, and audio recording. Their built APK sizes range from 29,960 KB to 35,204 KB, as shown in Table 2.

LLMs and corresponding agents. We evaluate three LLMs and their corresponding agents, as summarized in Table 3. Each agent was permitted to invoke jadx through MCP for APK inspection. The observed behavior may reflect the LLM, the provider’s safety policy, the corresponding agent, system instructions, context construction, and tool orchestration; we therefore do not attribute differences to the base LLMs alone.

Evaluation Procedure. We use application-level manifest metadata as the evaluated carrier. For each application–SRT-instance pair, the injector produces one APK variant, yielding 24 variants from six applications and four fixed SRT instances. Each variant is analyzed by the three LLMs and their corresponding agents in a fresh session with the same request and tool-access protocol.

Table 3: LLMs and their corresponding agents used in the main experiment.

LLM	Corresponding agent	Enabled tool	Runs
GPT-5.6 (Terra)	Codex	jadx MCP	24
Opus 5	Claude Code	jadx MCP	24
Grok 4.5	Codex	jadx MCP	24
Total			72

Table 4: Qualifying safety refusals for the four fixed SRT instances evaluated in the main experiment. Each SRT-instance cell contains one run for each of six applications. The four SRT columns represent individual trigger instances rather than category-level effectiveness estimates.

LLM and corresponding agent	C-SRT	B-SRT	R-SRT	N-SRT	Refusal Rate
GPT-5.6 + Codex	5/6	5/6	3/6	5/6	18/24 (75.0%)
Opus 5 + Claude Code	6/6	5/6	5/6	3/6	19/24 (79.2%)
Grok 4.5 + Codex	3/6	2/6	3/6	0/6	8/24 (33.3%)
Across evaluated LLMs and agents	14/18 (77.8%)	12/18 (66.7%)	11/18 (61.1%)	8/18 (44.4%)	45/72 (62.5%)

Metric. We count a run as a *safety refusal* when the agent explicitly declines the requested APK analysis for a safety-related reason or terminates before completing the task with such a rationale. Warnings or cautions followed by analysis are non-refusals. Technical failures, including timeouts, MCP disconnections, parsing errors, and unexplained product failures, are rerun in a fresh session; we retain the first valid terminal outcome for each experimental cell.

Our primary metric is the *Refusal Rate*, defined as the number of valid runs with a qualifying safety refusal divided by the total number of valid analysis runs. Valid runs exclude attempts that terminate solely because of a technical failure. We report descriptive counts and percentages; because each cell contributes one valid observation, we do not estimate run-to-run variance or perform population-level significance tests.

5.2 Results

RQ1: Refusal Effectiveness. Table 4 reports the main result. Across the six applications, four fixed SRT instances, and three LLM-agent configurations, 45 of 72 analysis runs ended in a qualifying safety refusal, for an overall refusal rate of 62.5%. In the evaluated non-adaptive setting, SRT-bearing manifest metadata therefore induced observable interruption in a substantial portion of the tested LLM-assisted reverse-engineering workflows.

Twenty-seven runs did not end in a qualifying safety refusal. The experiment therefore establishes interruption in the evaluated workflow, not prevention of reverse engineering: it does not measure the application logic or other protected information recovered in non-refusing runs.

Table 5: Text-control and sanitization ablation over two APKs and four fixed SRT instances. Column labels abbreviate the LLM-agent configurations used in the main experiment.

Condition	Claude	GPT	Grok	Total
Random text	0/8	0/8	0/8	0/24
Unsanitized SRT	5/8	5/8	2/8	12/24
Sanitized SRT	5/8	4/8	2/8	11/24

Finding 1. In the evaluated non-adaptive workflow, SRT-bearing APKs induced qualifying safety refusals in 45 of 72 analysis runs, providing feasibility evidence that safety-mediated interruption can affect LLM-assisted Android reverse engineering.

RQ2: Variation across Analysis Configurations and Selected SRT Instances. Refusal rates varied substantially across the evaluated LLM-agent configurations, ranging from 8/24 (33.3%) for Grok 4.5 with Codex to 19/24 (79.2%) for Opus 5 with Claude Code.

The four fixed SRT instances also produced different refusal outcomes across the tested systems. Because only one SRT instance is evaluated from each CBRN topic, these differences should not be interpreted as evidence that one CBRN category is inherently more effective than another. Instead, the results show that observed refusal behavior can vary with the particular SRT instance and the LLM-agent analysis configuration.

Differences among GPT-5.6, Opus 5, and Grok 4.5 should likewise be interpreted at the level of the complete analysis configuration rather than as effects of the base LLM alone, since provider safety policies, agent system instructions, context construction, and tool orchestration may all influence the terminal outcome.

Finding 2. Safety-refusal behavior varies across the evaluated LLM-agent configurations and individual SRT instances, indicating that the observed effect is configuration-dependent rather than universal.

RQ3: Effect of Sanitization. The ablation uses Fossify Clock and Fossify Music Player, the four fixed SRT instances, and the same three LLM-agent configurations. It compares random non-sensitive text, unsanitized source text, and sanitized SRTs. The random-text control is matched to the corresponding SRT in length, format, and encoding; it is reviewed as non-actionable and does not elicit a safety-related response during direct screening. All conditions use the same manifest carrier, request, tool access, and session protocol, yielding 24 analysis cells per text condition.

Table 5 reports the text-control and sanitization ablation. Random non-sensitive text produced no safety refusals in 24 runs, whereas unsanitized source text and sanitized SRTs produced 12/24 (50.0%) and 11/24 (45.8%) refusals, respectively.

Under the evaluated protocol, the length-matched random-text control suggests that the observed refusal behavior cannot be explained solely by the presence or length of additional manifest text. In this sample, sanitization retained 11 of the 12 refusals observed with unsanitized content while removing the operational details targeted by the sanitization procedure.

The ablation does not identify the precise semantic features responsible for refusal, establish statistical equivalence between

sanitized and unsanitized text, or show that arbitrary non-sensitive text can never produce refusal under other analysis conditions.

Finding 3. Sanitized SRTs may retain the ability to induce safety refusals while reducing the risk of embedding operational details that could support harmful use.

6 Discussion

LLMs and Reverse Engineering. LLMs and AI agents can make reverse engineering more useful for legitimate security work. They can help analysts interpret decompiled code, triage functions, and identify implementation details that merit further investigation. Such assistance can improve vulnerability discovery and remediation when used by application owners, authorized auditors, or security researchers.

The same capabilities can lower the cost of recovering proprietary application logic when used without authorization. For app vendors, this creates a code-protection problem that is distinct from the value of reverse engineering itself. SRTs extend app protection to the model-facing stage of LLM-assisted analysis by attempting to interrupt an LLM agent’s analysis of a distributed APK.

The mechanism is complementary to conventional protections rather than a replacement for them. Obfuscation and packing increase the intrinsic difficulty of recovering program structure or semantics, whereas SRTs target the artifact-to-context interaction introduced by LLM-assisted analysis. A practical protection stack could therefore combine conventional code-level protections with model-facing defenses, although the present pilot evaluates only the latter mechanism in isolation.

Adaptive Filtering and SRT Removability. The present study does not establish robustness against adaptive preprocessing. In particular, the evaluated manifest carrier is intentionally simple and observable, prioritizing experimental controllability over concealment. An analyst who recognizes the defense may remove suspicious metadata or apply deterministic preprocessing before extracted artifacts reach the downstream LLM. Such strategies fall outside the non-adaptive threat model evaluated in this study.

An LLM-based filtering stage creates a different interaction because the filtering model itself must process the embedded content. It is possible that the filtering model’s own safety mechanisms could affect this preprocessing step. However, we do not experimentally evaluate this behavior and therefore do not claim that SRTs resist LLM-based filtering. Future work should evaluate filter-aware attackers and investigate less conspicuous, distributed, or redundant placement of SRTs across multiple application artifacts.

Ethical Considerations. SRT research is dual use. A mechanism that helps vendors study defenses against unauthorized LLM-assisted reverse engineering could also impede automated malware analysis or other legitimate security workflows. We therefore limit the study to benign open-source applications. The prototype does not execute a payload, exploit an analysis tool, or attack an analyst’s environment. We omit operational CBRN text and report only high-level topic labels and aggregate measurements.

Any public release should be limited to the injector, benign build inputs, evaluation prompts, and non-actionable examples that pass an appropriate safety review. Operational or model-specific trigger text should not be released without a separate risk assessment.

Limitations and Future Work. This paper is a pilot study. Although the constructed corpus contains 40 accepted SRTs across four CBRN topics, the downstream APK-level experiment evaluates only four fixed SRT instances, one from each topic, across six applications and three LLM-agent configurations. The resulting measurements therefore establish feasibility for the evaluated instances rather than estimating the distribution of effectiveness across the full 40-item corpus or comparing the relative effectiveness of the four CBRN categories.

The six applications also come from a single open-source application family, each experimental cell contributes one valid terminal outcome, and the source corpus is screened in advance for direct safety-related responses. We therefore report descriptive measurements rather than population-level statistical conclusions. Future studies should evaluate larger numbers of trigger instances, repeated runs, additional application families, and broader LLM-agent configurations.

We measure safety refusal as an observable interruption outcome, not the amount of application logic, vulnerabilities, or other protected information recovered when analysis continues. Injected APKs build successfully, but we have not completed systematic runtime and regression testing. Future studies should combine refusal measurements with reverse-engineering outcomes at the task level and application-preservation tests.

The evaluated manifest carrier is visible and can be removed by an analyst who knows the defense. We do not evaluate adaptive analysts who filter application content, submit code in segments, modify prompts, switch models, or use local models without comparable safety guardrails. Provider safety policies, system prompts, model behavior, and context-handling strategies may also change over time. Future work should therefore evaluate adaptive workflows, compare the persistence and removability of different carriers, and track SRT behavior as relevant analysis systems evolve.

7 Related Work

LLM-assisted reverse engineering. LLMs have been evaluated on general and obfuscated-code analysis [7], and evidence from a human study shows that they can improve reverse-engineering triage and artifact recovery [2]. BINREX demonstrates agentic, tool-driven binary analysis [13], while work on Android examines obfuscation analysis with LLMs [8]. Nishizaka et al. characterize how software protection causes failures of LLM agents in CTF reverse engineering [14]. Together, these studies establish both the capability of LLM-assisted analysis and failure modes induced by software protection. Our work examines an Android artifact controlled by the defender and intended to induce a specific, observable outcome—a safety refusal—rather than measuring analysis capability or cataloging agent failures.

Active protection against LLM analysis. CHaT places traps, cloaks, and honeytokens in network assets to disrupt or detect LLM agents for penetration testing [1]. WebCloak turns webpages into self-protecting assets through structural obfuscation and harmless semantic misdirection [10]. Prompt injection has also been used defensively to reduce extraction of personal information with LLMs [12]. These systems show that an asset can influence the model that analyzes it, but they target network penetration testing, web scraping,

or information extraction. SRTs apply this principle to Android reverse engineering and evaluate a non-executable APK carrier whose intended effect is refusal triggered by safety guardrails rather than deception or adversarial instructions.

Manipulation through analyzed content. Prior work shows that content encountered during analysis can influence LLM-based systems. Prompt injection can redirect an application integrated with an LLM through content controlled by an attacker [11]. Crawford et al. show that strings recovered by reverse-engineering tools can manipulate analysis agents [5], while LogInject demonstrates passive manipulation through security logs [9]. Acoda applies semantics-preserving obfuscations to induce refusal or misinterpretation [15], whereas Familiar Pattern Attacks use small code edits that exploit abstraction bias to induce incorrect interpretation [3]. These studies motivate treating extracted application content as part of the model’s analysis surface. SRTs avoid instructions that redirect the task and do not depend on semantic misanalysis; instead, they expose sanitized policy-sensitive data to an existing safety boundary.

Safety refusal and anti-analysis. XSTest and OR-Bench document over-refusal on benign prompts that resemble unsafe requests [6, 16]. These benchmarks treat refusal as a model behavior to measure, rather than as a software-protection mechanism. A security report describes malicious software packages containing non-executing content intended to trigger safety policies and disrupt AI-assisted scanners [4], but provides no controlled evaluation. Our pilot studies this failure surface in a controlled Android reverse-engineering setting with benign apps, sanitized triggers, multiple analysis configurations, and explicit controls.

Taken together, prior work has studied LLM-assisted reverse engineering, manipulation through analyzed content, and safety refusal in largely separate lines of work. SRTs connect these directions by using safety refusal as a mechanism controlled by the defender for app protection.

8 Conclusion

LLMs and AI agents can lower the cost of recovering implementation knowledge from Android applications, creating a code-protection challenge for application vendors. We propose Safety-Refusal Triggers (SRTs), an active defense that embeds sanitized, policy-sensitive text in an APK and targets the artifact-to-context channel used by LLM-assisted reverse-engineering workflows.

Experiments with three LLM-agent configurations on six benign Android applications show that SRTs can induce observable safety-related interruption in the evaluated non-adaptive setting, with 45 of 72 main analysis runs ending in qualifying refusals. The random-text control further suggests that the observed behavior is associated with the safety-sensitive semantics of the injected content rather than merely the presence or length of additional manifest text.

These results establish the feasibility of safety-mediated interruption as a complementary Android app-protection primitive. They do not establish robustness against adaptive filtering or general prevention of reverse engineering. Future work should evaluate

broader trigger populations, more persistent and less conspicuous artifact carriers, repeated analysis runs, and adaptive analysis strategies.

References

- [1] Daniel Ayzenshteyn, Roy Weiss, and Yisroel Mirsky. 2025. Cloak, Honey, Trap: Proactive Defenses Against LLM Agents. In *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 8095–8114. <https://www.usenix.org/conference/usenixsecurity25/presentation/ayzenshteyn>
- [2] Zion Leonahenahe Basque, Samuele Doria, Ananta Soneji, Wil Gibbs, Adam Doupé, Yan Shoshitaishvili, Eleonora Losiouk, Ruoyu “Fish” Wang, and Simone Aonzo. 2026. Decompiling the Synergy: An Empirical Study of Human–LLM Teaming in Software Reverse Engineering. In *Network and Distributed System Security Symposium (NDSS)*. doi:10.14722/ndss.2026.240380
- [3] Shir Bernstein, David Beste, Daniel Ayzenshteyn, Lea Schönherr, and Yisroel Mirsky. 2026. Trust Me, I Know This Function: Hijacking LLM Static Analysis using Bias. In *Network and Distributed System Security Symposium (NDSS)*. doi:10.14722/ndss.2026.242066
- [4] Kirill Boychenko. 2026. Mini Shai-Hulud, Miasma, and Hades Worms Target Bioinformatics and MCP Developers via Malicious PyPI Wheels. Socket Security Research. <https://socket.dev/blog/mini-shai-hulud-miasma-and-hades-worms-target-bioinformatics-and-mcp-developers-via-malicious> Security vendor report, published June 8, 2026.
- [5] Brian Crawford, Justin Phillips, and Patrick McClure. 2026. Automatically Attacking Software Reverse Engineering AI Agents. arXiv:2605.30667 [cs.CR] doi:10.48550/arXiv.2605.30667
- [6] Justin Cui, Wei-Lin Chiang, Ion Stoica, and Cho-Jui Hsieh. 2025. OR-Bench: An Over-Refusal Benchmark for Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 11515–11542. <https://proceedings.mlr.press/v267/cui25a.html>
- [7] Chongzhou Fang, Ning Miao, Shaurya Srivastav, Jialin Liu, Ruoyu Zhang, Ruijie Fang, Asmita, Ryan Tsang, Najmeh Nazari, Han Wang, and Houman Homayoun. 2024. Large Language Models for Code Analysis: Do LLMs Really Do Their Job?. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 829–846. <https://www.usenix.org/conference/usenixsecurity24/presentation/fang>
- [8] Luca Ferrari, Marco Alecci, Jordan Samhi, Tegawendé F. Bissyandé, Jacques Klein, Mariano Ceccato, and Luca Verderame. 2026. Evaluating LLMs for Obfuscation Detection and Classification in Android Apps. arXiv:2606.14233 [cs.SE] doi:10.48550/arXiv.2606.14233
- [9] Rabimba Karanjai, Yang Lu, Hemanth Hegadehalli Madhavarao, Lei Xu, and Weidong Shi. 2026. Context Contamination in LLM Analysis of Network Security Logs: Poison with Passive Prompt Injection and Mitigation Evaluation. In *35th USENIX Security Symposium (USENIX Security 26)*. <https://www.usenix.org/conference/usenixsecurity26/presentation/karanjai>
- [10] Xinfeng Li, Tianze Qiu, Yingbin Jin, Lixu Wang, Hanqing Guo, Xiaojun Jia, Xiaofeng Wang, and Wei Dong. 2026. WebCloak: Characterizing and Mitigating Threats from LLM-Driven Web Agents as Intelligent Scrapers. In *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE, 436–455. doi:10.1109/SP63933.2026.00025
- [11] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and Benchmarking Prompt Injection Attacks and Defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 1831–1847. <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei>
- [12] Yupei Liu, Yuqi Jia, Jinyuan Jia, and Neil Zhenqiang Gong. 2025. Evaluating LLM-based Personal Information Extraction and Countermeasures. In *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 1669–1688. <https://www.usenix.org/conference/usenixsecurity25/presentation/liu-yupei>
- [13] Yuzhe Liu, Zhijie Liu, Zhengmin Yu, Shu Wang, Ling Jiang, Sen Nie, Shi Wu, Zhanyong Tang, and Yuan Zhang. 2026. Towards Generality: Task-Adaptive Binary Analysis via Semantic Retrieval and Verifiable Reasoning. In *35th USENIX Security Symposium (USENIX Security 26)*. <https://www.usenix.org/conference/usenixsecurity26/presentation/liu-yuzhe>
- [14] Ryutaro Nishizaka, Yudai Fujiwara, Takuya Shimizu, Kazushi Kato, and Yuichi Sugiyama. 2026. Towards LLM-Resistant Software Protection: Agent Failure Patterns in CTF Reverse Engineering. In *Workshop on Binary Analysis Research (BAR)*. <https://www.ndss-symposium.org/ndss-paper/auto-draft-657/>
- [15] Hongzhou Rao, Zikan Dong, Yanjie Zhao, Haodong Li, and Haoyu Wang. 2026. Acoda: Adversarial Code Obfuscation for Defending against LLM-based Analysis. arXiv:2606.11755 [cs.SE] doi:10.48550/arXiv.2606.11755
- [16] Paul Röttger, Hannah Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. 2024. XSTest: A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human*

